WEB FRONTEND DEVELOPMENT

Case Study 3 – Semester 2

FERNANDO RODRÍGUEZ MORAL
Fontys - ICT

Table of Contents

1. Analysis 2
2. Design 3
   2.1. Flask Application Structure 3
   2.2. Flask Routes 3
   2.3. Wireframe – Dashboard Layout 4
3. Realization 5

1. Analysis

The web interface must provide authorized staff with a clear, accessible view of system monitoring data without requiring CLI access or direct server knowledge. The following key information and functionalities have been identified:

Information to Display

Category	Data Points
System Overview	Hostname, OS, uptime, last updated timestamp
CPU	Usage percentage, core count, per-core load
Memory	Total, used, available RAM (%), swap usage
Disk	Per-partition usage, read/write I/O stats
Network	Bytes sent/received, active interfaces, IP addresses
Services	Status of key services (running / stopped / unknown)
Containers	Container name, status, CPU %, memory usage (Week 3+)
Alerts	Threshold breaches, recent warnings

User Roles (Conceptual)

Role	Access Level
Admin	Full dashboard + configuration access
Staff	Read-only monitoring dashboard

2. Design

2.1. Flask Application Structure

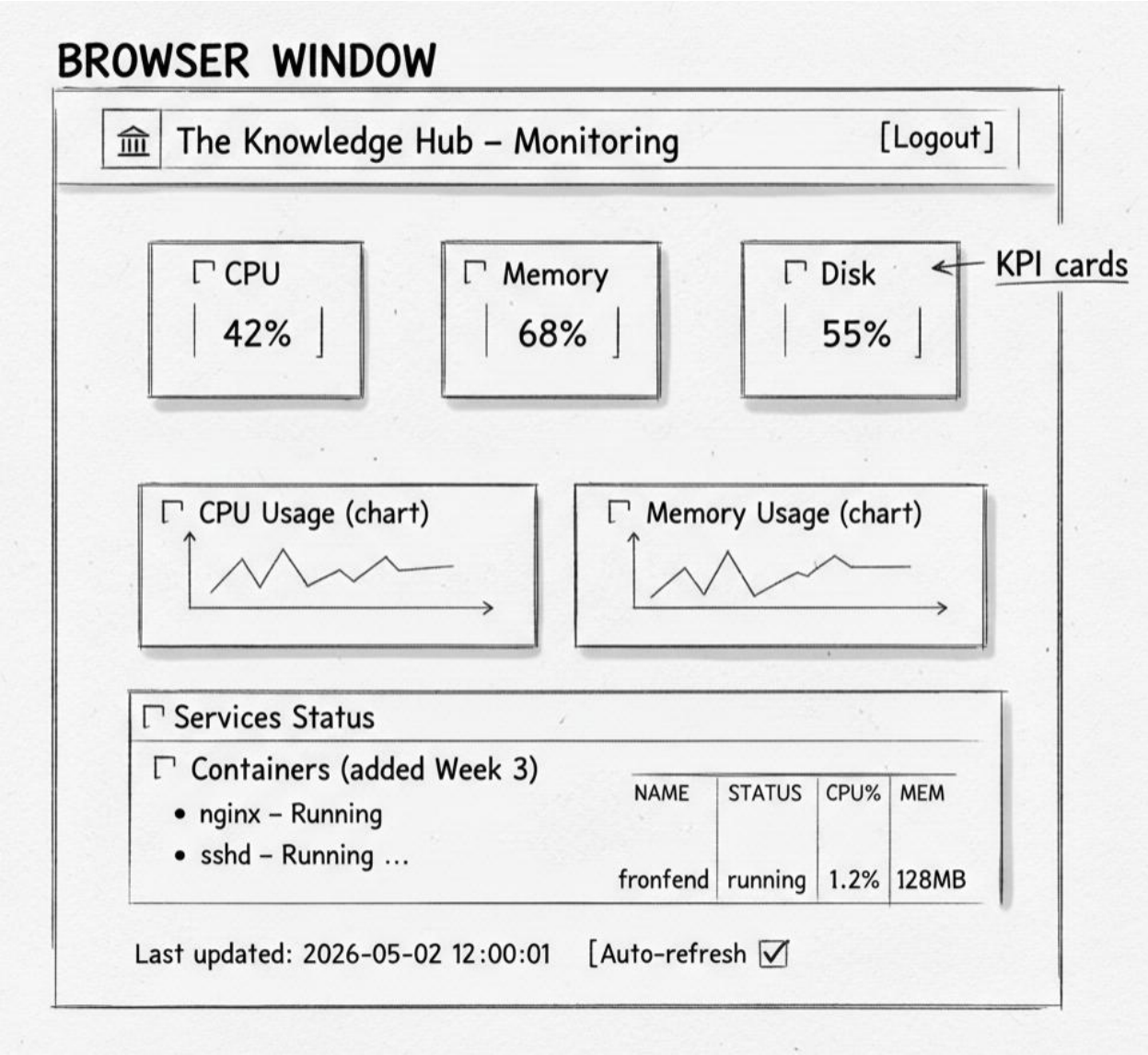
```
monitoring-app/
├── app/
│   ├── __init__.py          # App factory, register blueprints
│   ├── routes.py            # Flask routes
│   ├── auth.py              # Entra ID authentication logic (Task 4)
│   ├── collector.py         # Data collection calls to backend/API
│   ├── templates/
│   │   ├── base.html        # Base layout (navbar, footer)
│   │   ├── dashboard.html   # Main monitoring dashboard
│   │   ├── login.html       # Login / auth redirect page
│   │   └── error.html       # Error page (404, 500)
│   └── static/
│       ├── css/
│       │   └── style.css
│       └── js/
│           └── dashboard.js # Auto-refresh, charts
├── config.py                # App configuration (env vars)
├── requirements.txt
└── Dockerfile               # Added in Task 5

```

2.2. Flask Routes

Method	Route	Description	Auth Required
GET	/	Redirect to /dashboard or /login	No
GET	/login	Login page / Entra ID redirect	No
GET	/auth/callback	OAuth2 callback from Entra ID	No
GET	/dashboard	Main monitoring dashboard	Yes
GET	/api/metrics	JSON endpoint – system metrics	Yes
GET	/api/containers	JSON endpoint – container metrics	Yes
GET	/logout	Clear session and redirect to login	Yes

2.3. Wireframe – Dashboard Layout



3. Realization

config.py

```
import os

class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-key")
    BACKEND_API_URL = os.environ.get("BACKEND_API_URL", "http://backend:5001")
    ENTRA_CLIENT_ID = os.environ.get("ENTRA_CLIENT_ID", "")
    ENTRA_CLIENT_SECRET = os.environ.get("ENTRA_CLIENT_SECRET", "")
    ENTRA_TENANT_ID = os.environ.get("ENTRA_TENANT_ID", "")
    ENTRA_REDIRECT_URI = os.environ.get("ENTRA_REDIRECT_URI",
"http://localhost:5000/auth/callback")

```

app/__init__.py

```
from flask import Flask
from config import Config

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    from app.routes import main
    app.register_blueprint(main)

    return app

```

app/routes.py

```
from flask import Blueprint, render_template, redirect, url_for, session
from app.collector import get_metrics, get_containers

main = Blueprint("main", __name__)

def login_required(f):
    from functools import wraps
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user" not in session:
            return redirect(url_for("main.login"))
        return f(*args, **kwargs)
    return decorated

@main.route("/")
def index():
    return redirect(url_for("main.dashboard"))

@main.route("/login")
def login():
    return render_template("login.html")

@main.route("/dashboard")
@login_required
def dashboard():
    metrics = get_metrics()
    containers = get_containers()
    return render_template("dashboard.html", metrics=metrics, containers=containers)

```

```
@main.route("/api/metrics")
@login_required
def api_metrics():
    from flask import jsonify
    return jsonify(get_metrics())

@main.route("/api/containers")
@login_required
def api_containers():
    from flask import jsonify
    return jsonify(get_containers())

@main.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("main.login"))

```

app/collector.py

```
import requests
from flask import current_app

def get_metrics():
    try:
        url = current_app.config["BACKEND_API_URL"] + "/metrics"
        response = requests.get(url, timeout=5)
        return response.json()
    except Exception as e:
        return {"error": str(e)}

def get_containers():
    try:
        url = current_app.config["BACKEND_API_URL"] + "/containers"
        response = requests.get(url, timeout=5)
        return response.json()
    except Exception as e:
        return {"error": str(e)}

```

app/templates/base.html

```
xml
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>The Knowledge Hub – Monitoring</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <header>
        <nav>
            <span class="brand">📊 Knowledge Hub Monitoring</span>
            {% if session.get('user') %}
                <span class="user">{{ session['user']['name'] }}</span>
                <a href="{{ url_for('main.logout') }}">Logout</a>
            {% endif %}
        </nav>
    </header>
    <main>
        {% block content %}{% endblock %}
    </main>
```

```
    <footer>
        <p>Last updated: <span id="last-updated">–</span></p>
    </footer>
    <script src="{{ url_for('static', filename='js/dashboard.js') }}"></script>
</body>
</html>

```

app/templates/dashboard.html

```
xml
{% extends "base.html" %}
{% block content %}

<section class="kpi-cards">
    <div class="card">
        <h3>CPU</h3>
        <p class="value">{{ metrics.cpu.percent }}%</p>
    </div>
    <div class="card">
        <h3>Memory</h3>
        <p class="value">{{ metrics.memory.percent }}%</p>
    </div>
    <div class="card">
        <h3>Disk</h3>
        <p class="value">{{ metrics.disk.percent }}%</p>
    </div>
</section>

<section class="services">
    <h2>Services</h2>
    <table>
        <thead><tr><th>Service</th><th>Status</th></tr></thead>
        <tbody>
        {% for service in metrics.services %}
            <tr>
                <td>{{ service.name }}</td>
                <td class="{{ 'running' if service.status == 'running' else 'stopped' }}">
                    {{ service.status }}
                </td>
            </tr>
        {% endfor %}
        </tbody>
    </table>
</section>

<section class="containers">
    <h2>Containers</h2>
    <table>
        <thead><tr><th>Name</th><th>Status</th><th>CPU %</th><th>Memory</th></tr></thead>
        <tbody>
        {% for c in containers %}
            <tr>
                <td>{{ c.name }}</td>
                <td>{{ c.status }}</td>
                <td>{{ c.cpu_percent }}</td>
                <td>{{ c.memory_usage }}</td>
            </tr>
        {% endfor %}
        </tbody>
    </table>
</section>

{% endblock %}
```

requirements.txt

```
flask>=3.0
requests>=2.31
msal>=1.28

```

static/js/dashboard.js

```
// Auto-refresh metrics every 30 seconds
setInterval(() => {
    fetch("/api/metrics")
        .then(r => r.json())
        .then(data => {
            document.querySelector(".kpi-cards .card:nth-child(1) .value").textContent = data.cpu.percent + "%";
            document.querySelector(".kpi-cards .card:nth-child(2) .value").textContent = data.memory.percent + "%";
            document.querySelector(".kpi-cards .card:nth-child(3) .value").textContent = data.disk.percent + "%";
        });
    document.getElementById("last-updated").textContent = new Date().toLocaleString();
}, 30000);

document.getElementById("last-updated").textContent = new Date().toLocaleString();

```